Hands-On Machine Learning

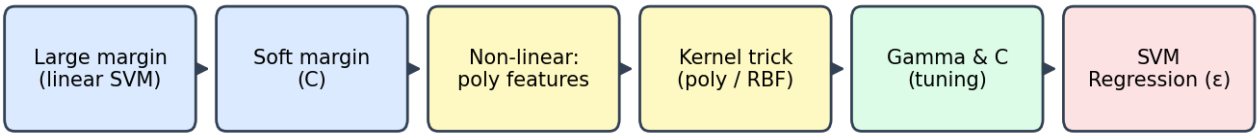
Chapter 5 — Support Vector Machines

Margin · Kernel Trick · RBF · SVM Regression

ملخص كشكولك + تكملة النقط الناقصة + رسومات وأكواد للمراجعة

من كشكولك | إضافة لإكمال النقص | انتبه / تصحيح

كل الأرقام في مربعات **Output (real run)** الخضرا طلعت من تشغيل الكود فعلًا (داتا Iris الحقيقية، وداتا moons المولّدة بنفس إعدادات الكتاب). الرسومات اللي عليها (synthetic) داتا مولّدة للتوضيح فقط.



خريطة الشابتر: كل خطوة بتحل مشكلة اللي قبلها.

1 يعني إيه SVM؟

من كشكولك

يعني إيه SVM؟ هو SVM Model أصلًا ممكن يُستخدم في:

- **Classification:** تحديد الفئة التي تنتمي إليها العينة.
- **Regression:** توقع قيمة رقمية.
- **Novelty Detection:** اكتشاف عينة غير معتادة (Ch 9).

← الكتاب بيشير إلى أن SVM بيكون **مفيد خصوصًا مع الـ Datasets الصغيرة والمتوسطة**، وخاصة في مشاكل Non linear classification، لكنه **لا يتوسع جيدًا** مع أعداد ضخمة من العينات في بعض تطبيقاته.

إضافة

أسباب ده بنشوفها بالأرقام في القسم 5 (جدول التعقيد): الـ `SVC` الأصلي وقت تدريبه بيزيد **أسرع بكتير** من عدد العينات. ولو الداتا ضخمة جدًا الكتاب بيقترح بدائل: `LinearSVC` (خطي بس) أو `SGDClassifier`.

نقطة بنحتاجها بعدين: الـ SVM في الأصل **binary classifier**. لو عندك أكتر من فئتين (Multiclass) scikit-learn بتحوّله تلقائيًا (OvR أو OvO) زي ما شفنا في شابتر 3؛ `SVC` بتستخدم OvO و`LinearSVC` بتستخدم OvR.

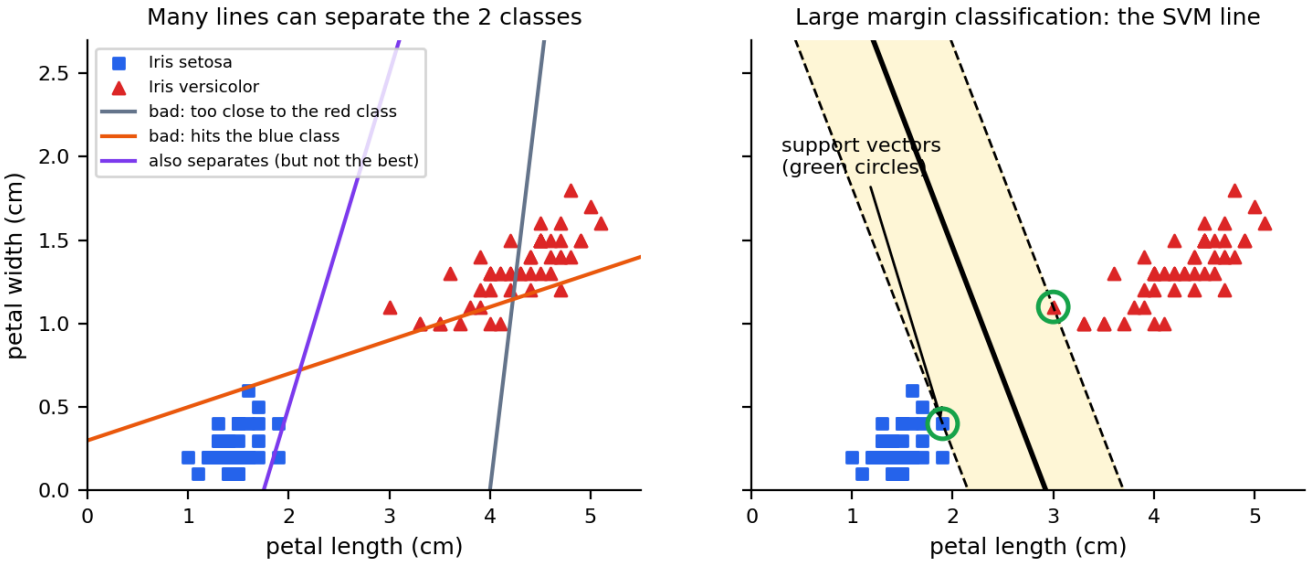
2 الـ Linear SVM: Large Margin Classification

من كشكولك

Large Margin classification: تخيل إن عندك Dataset فيها فئتين وعايز تفصل بينهم بخط مستقيم، ممكن تلاقي أكتر من خط يفصل الفئتين بشكل صحيح لكن مش كل الخطوط بنفس الجودة. مثلًا خط بيعدي قريب جدًا من بعض النقاط، تغيير بسيط في البيانات ممكن يخليه يصنف عينات جديدة بشكل سيء.

هنا الـ SVM عنده **فكرة مختلفة:** مش كفاية إن الخط يفصل الفئتين، عايزين **أكبر مسافة ممكنة** بين الحد الفاصل وأقرب نقاط من الفئتين.

← المساحة بين الخطين المتوازيين حول الحد الفاصل اسمها (Margin) والطريقة دي اسمها (Large Margin classification).



داتا Iris الحقيقية (setosa مقابل versicolor): شمال: خطوط كتير تفصل الفئتين. يمين: خط الـ SVM بيختار الخط اللي بيعمل أعرض شارع (الأصفر). الدوايز الخضرا = الـ support vectors.

من كشكولك

يعني اختيار الحد الفاصل الذي يحقق **أكبر Margin ممكن**، مع مراعاة القيود الخاصة بالموديل. طبعًا الـ Support vectors هي نقاط التدريب الموجودة على حوافي الـ Margin أو التي تؤثر في تحديدها.

الفكرة المهمة: إن الحدود الفاصلة بتتحدد أساسًا بواسطة هذه النقاط، إضافة نقاط أخرى بعيدة عن الـ Margin عادةً لا تغيّر الحد الفاصل في الحل نفسه.

إذن:

- **Decision Boundary:** الحد الذي يفصل بين الفئتين.
- **Margin:** المساحة حول الحد الفاصل.
- **Support Vectors:** النقاط المؤثرة في تحديد حدود الـ Margin. ⇒ وده جوهر اسم Support Vector Machine.

إضافة

الفكرة بمعادلات (من الكتاب): الموديل بيحسب score لكل عينة، والـ decision boundary هو المكان اللي الـ score فيه = 0، والـ margin بتتحدد بحدود `score = ±1` :

$$\text{score} = \mathbf{w}^T\mathbf{x} + b\ , \qquad \hat{y} = 1 \text{ if score} \geq 0, \quad 0 \text{ otherwise}$$

$$\text{margin width} = \frac{2}{\|\mathbf{w}\|}$$

يعني **تكبير الـ margin = تصغير** $\|\mathbf{w}\|$. ده بالظبط اللي بيعمله التدريب: بيدوّر على أصغر $\|\mathbf{w}\|$ بشرط إن كل النقاط تبقى برا الشارع (hard margin، القسم 4).

وبالأرقام الحقيقية على Iris: بعد التدريب لقيت `w = [1.294, 0.824]` و`b = -3.788`، يعني عرض الـ margin = 2 / ‖w‖ = **1.304** سم، وعدد الـ support vectors = **2** بس (واحد من كل فئة) من أصل 100 نقطة!

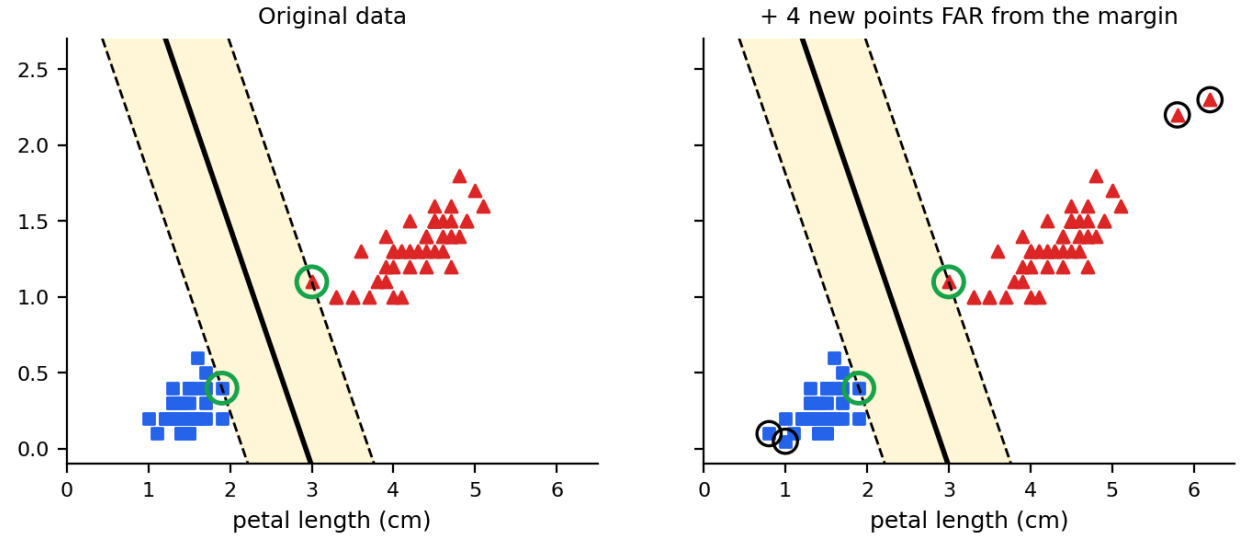
من كشكولك

ملاحظة من كشكولك (جربتها فعلًا): إضافة نقاط بعيدة عن الـ margin ما بتغيّرش الحد الفاصل.

```
svm = SVC(kernel="linear", C=1e9).fit(X, y)                  # original data
svm_far = SVC(kernel="linear", C=1e9).fit(X_with_4_far_points, y_with_4)

print(svm.coef_, svm.intercept_)
print(svm_far.coef_, svm_far.intercept_)
```

```
Output (real run)
[[1.294 0.824]] [-3.7882]
[[1.294 0.824]] [-3.7882]      # identical!
```



تجربة حقيقية: أضفنا 4 نقاط (الدوايز السودا) بعيدة عن الـ margin، والخط والـ margin ما اتغيّروش خالص (نفس `w` و`b`). الـ support vectors (الدوايز الخضرا) هي اللي بتقرر.

إضافة

ملحوظة مهمة: ده صحيح للنقاط البعيدة عن الـ margin. لكن نقطة جديدة **على** الـ margin أو **جواه** (أو في الجانب الغلط) بتبقى support vector جديدة وبتغيّر الحل. وده اللي حصل في الـ soft margin (القسم 4).

3 ليه لازم نعمل Feature Scaling؟

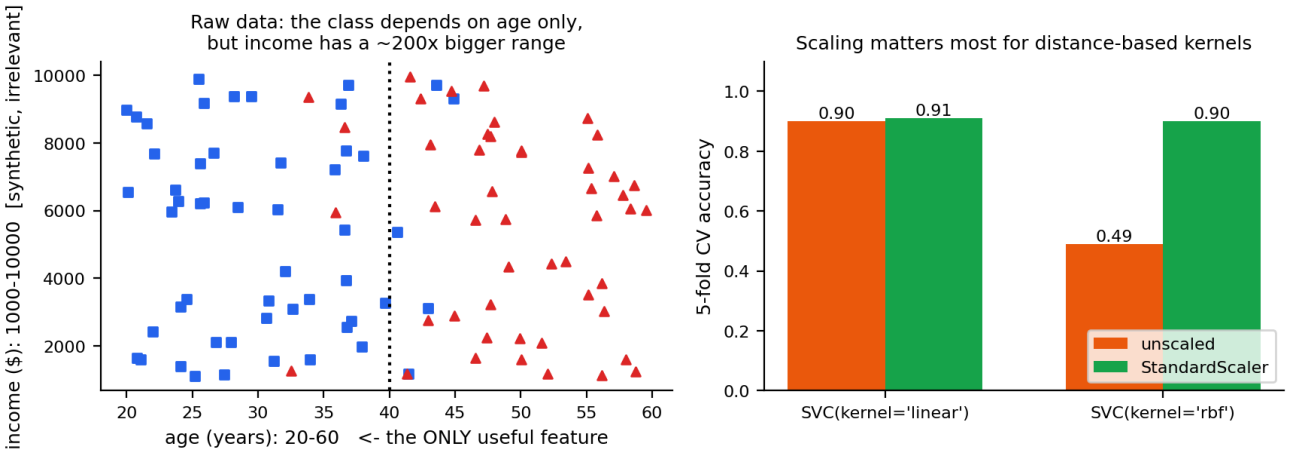
من كشكولك

الـ SVM **حساس جدًا** لاختلاف مقاييس الـ features. ← الـ Feature الكبيرة في المقياس ممكن تؤثر بشدة على حساب المسافات والـ Margin فتتغير النتيجة حسب وحدات القياس نفسها. عشان كده غالبًا بنعمل Scaling قبل تدريب الـ SVM.

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score

X_scaled = StandardScaler().fit_transform(X)      # X = [age (20-60), income (1000-10000)]
for kernel in ["linear", "rbf"]:
    print(kernel, cross_val_score(SVC(kernel=kernel), X, y, cv=5).mean(),
                  cross_val_score(SVC(kernel=kernel), X_scaled, y, cv=5).mean())
```

```
Output (real run)
linear 0.90 0.91      # unscaled  scaled
rbf    0.49 0.90
```



داتا synthetic مبنية عمدًا: الـ class بيعتمد على `age` بس، و`income` عمود مالوش علاقة بس مداه أكبر بـ 200 مرة. الـ RBF SVM بدون scaling دقته 49% (زي التخمين!)، ومعاه 90%.

إضافة

انتبه للتفصيلة دي (طلعت من التجربة): تأثير الـ scaling على الـ **RBF** (والـ polynomial) كان ضخم (من 0.49 لـ 0.90) لأن الـ kernel كله مبني على **المسافة**، فالـ feature الكبيرة بتبلع الباقي. لكن على الـ **linear** الفرق كان صغير (0.90 مقابل 0.91) لأن الـ SVM الخطي ممكن "يعوّض" بـ weight صغير للعمود الكبير. ده مش معناه نتجاهل الـ scaling في الـ linear: بيأثر برضه على الـ margin وسرعة التقارب، فالقاعدة الآمنة: **دايمًا** `StandardScaler` **قبل أي SVM**.

والقاعدة المعتادة: `fit` على الـ training set بس، وبعدين `transform` على الباقي (وده اللي الـ `make_pipeline` بيعمله تلقائيًا).

4 Soft Margin و Hard Margin

من كشكولك

تخيّل إننا اشترطنا إن كل عينة لازم تكون: **(1)** في الناحية الصحيحة من الحد الفاصل، **(2)** خارج الـ Margin (من غير أي انتهاكات). ⇒ دي اسمها **Hard Margin classification**.

لكن فيه مشكلتين:

1. لو البيانات **غير قابلة للفصل** بخط مستقيم، مش هيكون فيه حل يحقق الشروط دي.
2. وجود **outlier** واحد ممكن يغيّر الحد الفاصل بشدة أو يجعل الفصل مستحيل.

بالتالي التشدد الكامل **مش مناسب لكل البيانات الواقعية**.

إضافة

$$\min_{\mathbf{w},\,b} \frac{1}{2}\mathbf{w}^T\mathbf{w} \quad \text{s.t.} \quad t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 \quad \forall i$$

(المعادلة: أصغر `‖w‖` بشرط إن كل نقطة `t(i)=±1` تحقق `score ≥ 1`، يعني برا الشارع تمامًا.)

من كشكولك

Soft Margin classification: بدل ما نصر على إن كل النقاط تحقق الشروط، **نسمح ببعض الانتهاكات**: الانتهاك ممكن يحصل لما عينة تدخل جوه الـ Margin أو حتى تقع في الجانب الخطأ من الحد الفاصل.

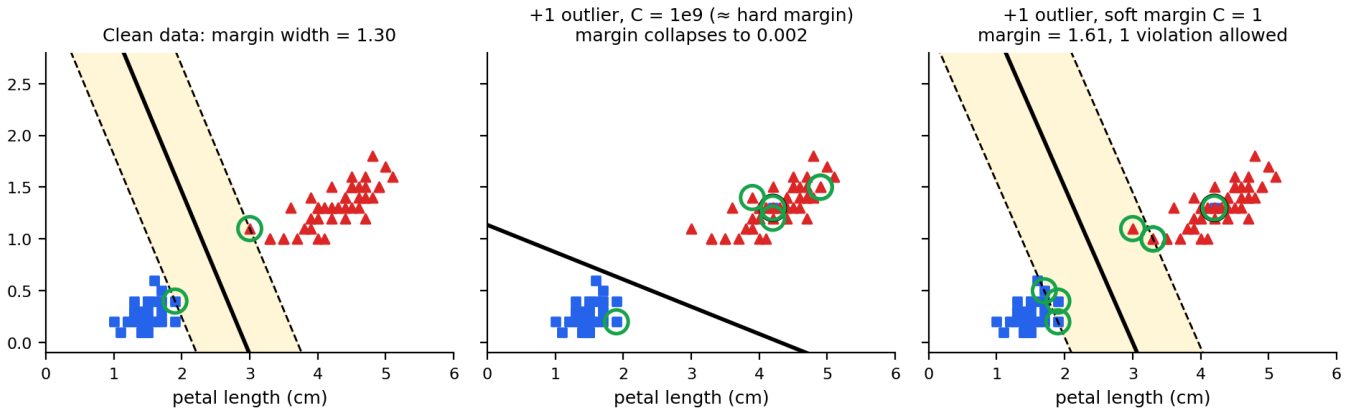
الهدف يصبح **توازن** بين تكبير الـ Margin وتقليل عدد أو مقدار الانتهاكات. وده اسمه Soft Margin classification.

إضافة

$$\min_{\mathbf{w},\,b,\,\zeta} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)} \quad \text{s.t.} \quad t^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)}, \quad \zeta^{(i)} \geq 0$$

الـ `ζ(i)` (zeta) = مقدار انتهاك العينة i (صفر لو مفيش انتهاك). والـ `C` بيوزن "تكلفة الانتهاكات" مقابل "عرض الـ margin". نفس الفكرة بصيغة الـ loss هي الـ **hinge loss**:

$$\text{hinge loss} = \max(0,\ 1 - t \cdot \text{score}) \qquad (t = \pm 1)$$



تجربة حقيقية على Iris: شمال: داتا نضيفة عرض الـ margin = 1.30. نص: أضفنا outlier واحد وسط الفئة التانية (الفصل مبقاش ممكن): مع C=1e9 (قريب من hard) الـ margin انهار لـ 0.002. يمين: نفس الداتا بـ soft margin (C=1): margin = 1.61 وانتهاك واحد مسموح.

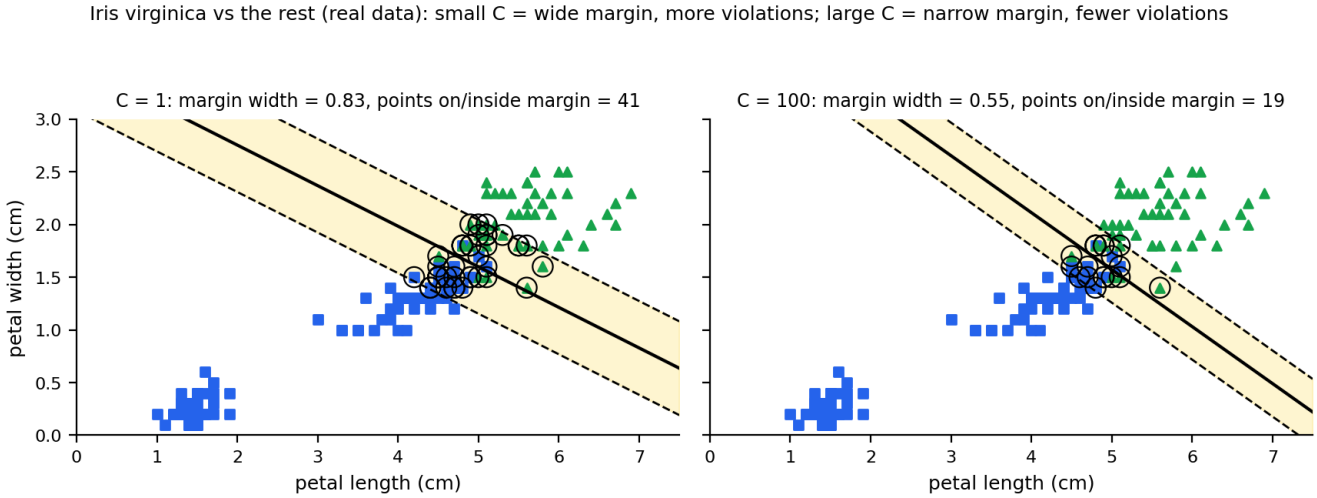
من كشكولك

دور الـ `C` Hyperparameter: في scikit-learn الـ `C` بيتحكم في التوازن ده:

- `C` **صغيرة:** نسمح بانتهاكات أكتر ونعطي الهامش أكبر (اتساع الـ Margin).
- `C` **كبيرة:** نعاقب الانتهاكات بقوة أكبر ونحاول تصنيف نقاط التدريب بصورة أكتر تشددًا.

⇒ وبالتالي **تقليل الـ `C` يعمل Regularization أقوى** وده ممكن يقلل الـ overfitting، لكن لو قللتها بشكل مبالغ ممكن الموديل يعمل underfitting.

ملحوظة: مش معنى إن `C` أكبر إن الموديل أفضل، وإنما الاختيار المناسب يعتمد على البيانات ونتائج التقييم.



داتا Iris الحقيقية (virginica مقابل الباقي): C=1: margin أعرض (0.83) وعدد النقاط على/جوه الـ margin = 41. C=100: margin أضيق (0.55) والنقاط = 19.

إضافة

وده تأكيد لملحوظتك بالأرقام الحقيقية: دقة الـ cross-validation (5 folds) لـ `LinearSVC` على نفس الداتا بقيم `C` مختلفة:

C	0.001	0.01	0.1	1	10	100	1000
CV accuracy	0.807	0.867	0.967	0.953	0.953	0.953	0.953

لما `C` صغير جدًا (0.001) الموديل بيعمل **underfitting** (دقة 0.81)، وأحسن قيمة طلعت `C=0.1` (0.967)، وبعدها الدقة بتثبت. يعني زيادة `C` فوق نقطة معينة مبتفيدش. الاختيار بيتعمل بالـ cross-validation (`GridSearchCV`) مش بالتخمين.

5 تطبيق Linear SVM باستخدام scikit-learn

من كشكولك

الكتاب بيستخدم Iris Dataset لتصنيف زهرة **Iris Virginica** مقابل باقي الزهور:

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2)            # Iris virginica -> True, any other class -> False

svm_clf = make_pipeline(StandardScaler(), LinearSVC(C=1, random_state=42))
svm_clf.fit(X, y)
```

من كشكولك

`y = (iris.target == 2)` دي بتحوّل الـ Target إلى Binary labels: Iris Virginica → `True`، أي فئة أخرى `False`.

ليه `random_state=42` ؟ بعض خوارزميات الـ ML بتعتمد على عمليات عشوائية في البداية (زي توزيع الأوزان العشوائية أو تقطيع/خلط البيانات). لما بتحدد رقم ثابت زي `random_state` بيخلي الكمبيوتر يختار أرقام عشوائية بطريقة **ثابتة ومحفوظة**، بالتالي لو حبيت تغيّر قيمة الـ `C` وتشوف تأثيرها، `random_state` بيخليك متأكد إن أي تغيّر في دقة الموديل سببه تعديل الـ `C` فعلًا مش بسبب إن الكمبيوتر خلط البيانات أو بدأ بطريقة عشوائية مختلفة.

الـ Pipeline: `Input Features` → `StandardScaler` → `LinearSVC` → `Prediction`.

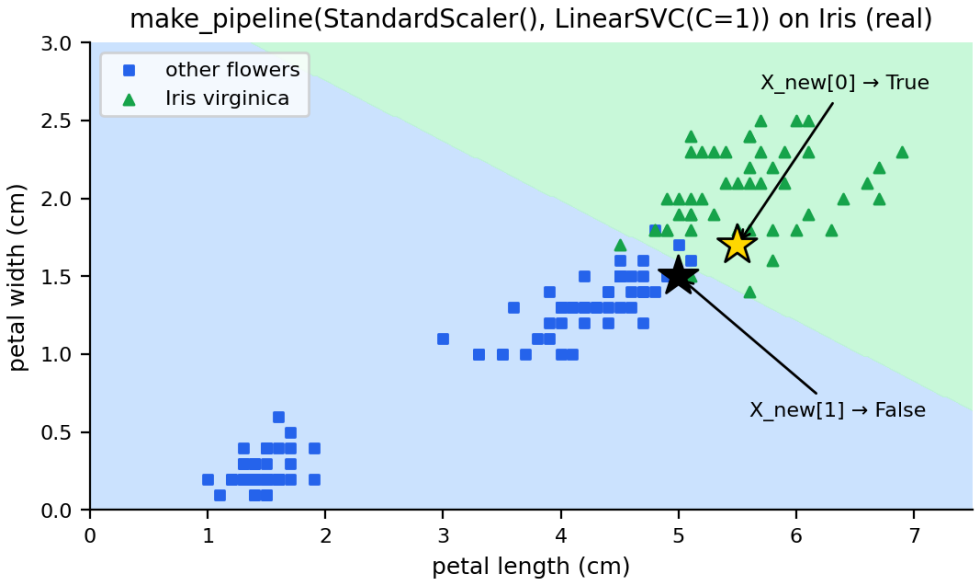
```
X_new = [[5.5, 1.7], [5.0, 1.5]]
print(svm_clf.predict(X_new))
```

Output (real run)

```
[True, False]      # train accuracy on Iris = 0.96
```

من كشكولك

- الـ `X_new` دي أرقام: القيم دي **مش متجابة من الـ Dataset**، افتراضية المفروض هو الكتاب لتجربة عشان يوريك بيها إزاي الموديل بيشتغل.
- الشكل مكتوب `[petal length, petal width]` : **2 features** لأن الموديل اتدرّب على الـ 2 دول، فلازم أي عينة جديدة تدخل له تحتوي على القيمتين دول **بنفس الترتيب**.
- يعني: الوردة الأولى تبع Iris Virginica، والوردة التانية صنفت على إنها `False`.
- في مشاريع حقيقية `X_new` دي بتبقى عبارة عن بيانات جديدة جاتلك (مثلًا صور تم معالجتها أو قياسات حقيقية لجهاز استشعار) وعايز الموديل يحكم عليها وهو مشافهاش قبل كده.



منطقة القرار الحقيقية للـ pipeline على Iris، والنجمتين = `X_new`.

```
print(svm_clf.decision_function(X_new))
```

```
Output (real run)
[ 0.6616 -0.2204]      # the book prints [ 0.66163411 -0.22036063 ]
```

من كشكولك

`svm_clf.decision_function(X_new)` ترجع **scores** للعينة بالنسبة إلى الحد الفاصل: مثلًا `[0.66, -0.22]`:

- **score موجب** ← العينة في جهة الـ Positive.
- **score سالب** ← العينة في جهة الـ Negative.
- **score قريب من الصفر** ← العينة قريبة من الحد الفاصل.

← الـ score هنا **مقياس لموقع العينة بالنسبة إلى الحد الفاصل**، وليس Probability.

← `LinearSVC` مفيهاش `predict_proba` عكس `LogisticRegression`، لا توفّر تقدير احتمالية مباشرة. الكتاب يوضح إنك لو استخدمت `SVC(probability=True)` فالموديل بيضيف عملية لتقدير الاحتمالات من الـ decision scores باستخدام **نموذج إضافي**، وده بيزود تكلفة التدريب.

```
svc = make_pipeline(StandardScaler(), SVC(kernel="linear", C=1, probability=True, random_state=42))
svc.fit(X, y)
print(svc.predict_proba(X_new))          # [P(False), P(True)] for each flower
```

```
Output (real run)
[[0.188 0.812]
 [0.637 0.363]]
# hasattr(LinearSVC, 'predict_proba') = False
```

إضافة

النتيجة منطقية: الوردة الأولى (score موجب) احتمال إنها virginica = **81%**، والتانية (score سالب) = **36%**. الاحتمالات دي بتتحسب بنموذج إضافي (Platt scaling + cross-validation داخلي) مش من الـ SVM نفسه، فهي **تقدير** مش حقيقة. (على الداتا الصغيرة دي الفرق في الوقت مش ملحوظ: 1.4 مللي ثانية مقابل 1.9؛ لكن على داتا كبيرة بيفرق.)

وفيه 3 طرق لتدريب SVM خطي في scikit-learn، والجدول ده من الكتاب (مع تجربة وقت حقيقية تحته):

الكلاس	التعقيد الزمني	Out-of-core	محتاج Scaling؟	Kernel trick
`LinearSVC`	O(m × n)	لأ	أيوه	لأ
`SVC`	O(m² × n) لـ O(m³ × n)	لأ	أيوه	**أيوه**
`SGDClassifier`	O(m × n)	**أيوه**	أيوه	لأ

(`m` = عدد العينات، `n` = عدد الـ features.) وتجربة وقت حقيقية على moons (ثواني، الجهاز بمعالج واحد):

عدد العينات m	LinearSVC	SVC (rbf)	نسبة SVC / Linear
500	0.0015	0.0023	2x
2000	0.0015	0.0119	8x
6000	0.0028	0.0773	27x

لما m زادت 12 مرة (500 ← 6000) وقت `LinearSVC` زاد حوالي 3 مرات بس، لكن وقت `SVC` زاد حوالي 40 مرة. ده تقريبًا اللي الكتاب بيقصده بـ "لا يتوسع جيدًا مع أعداد ضخمة من العينات".

6 Nonlinear SVM Classification

من كشكولك

ماذا لو البيانات لا يمكن فصلها بخط مستقيم؟ الـ linear SVM ممكن يكون غير مناسب لو شكل البيانات غير خطي. كنا بنضيف Features جديدة مثل `x²` عشان الموديل يقدر يمثل علاقة منحنية (Polynomial Regression في ch 4). **نفس الفكرة ممكن نستخدمها مع SVM.**

تخيّل عندك Feature واحد `x` وعلاقة الفصل معقدة، نضيف Feature جديدة: `x2 = x1²` . بالتالي البيانات أصبحت متمثلة بـ Feature الأصلية و Feature مشتقة منها.

الفكرة العامة: تحوّل البيانات إلى Features أكتر تعبيرًا ثم نستخدم `linear SVM` عليها.

من كشكولك

تنفيذ Polynomial features مع linear SVM

الكتاب بيستخدم `make_moons()` وهي داتا تجريبية لهلالين متداخلين:

```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import LinearSVC

X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
polynomial_svm_clf = make_pipeline(PolynomialFeatures(degree=3),
                                   StandardScaler(),
                                   LinearSVC(C=10, max_iter=10000, random_state=42))
polynomial_svm_clf.fit(X, y)
print(polynomial_svm_clf.score(X, y))
```

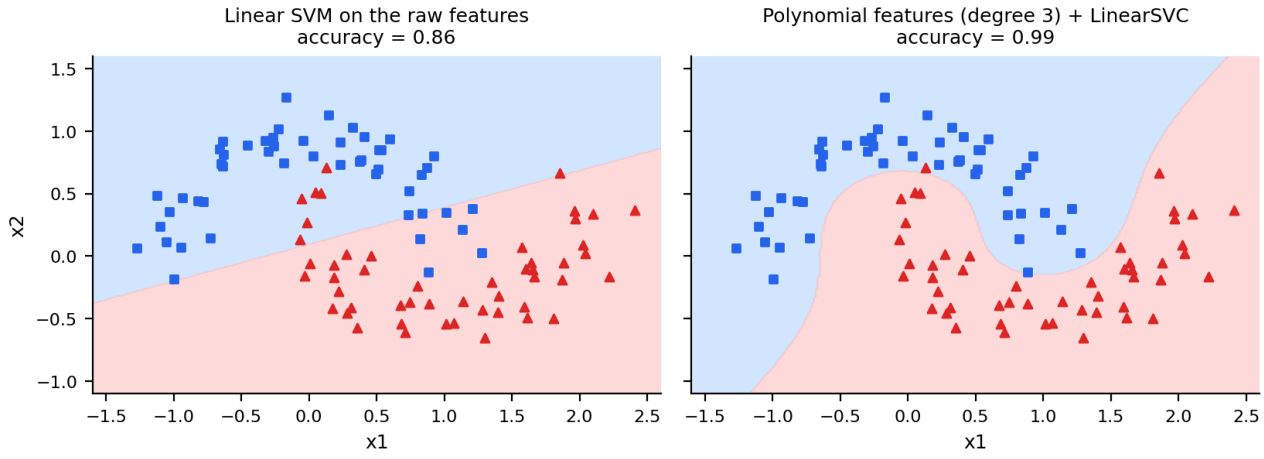
```
Output (real run)
0.99      # (a plain linear SVM on the raw features: 0.86)
```

من كشكولك

الـ Pipeline يعمل: **(1)** تضيف Features من الدرجة الثالثة. **(2)** Scaling للـ Features الجديدة. **(3)** تدريب `linear` `SVM` على البيانات بعد التحويل.

لكن في مشكلة: كلما زاد Degree أو عدد الـ Features الأصلية، عدد Features الجديدة ممكن يكبر جدًا ويبطّئ التدريب ويحتاج ذاكرة أكتر، وهنا ياتي دور الـ Kernel Trick.

`max_iter=10000` : هي الحد الأقصى لخطوات التكرار اللي الخوارزمية مسموح لها. الـ default بس 1000، لكن لما دخلنا `PolynomialFeatures(degree=3)` الـ Features بقت كتيرة ومعقدة وده بيخلي الخوارزمية تحتاج خطوات أكتر عشان تضبط الأوزان وتتأكد إنها وصلت لأفضل تقارب (Convergence).



داتا moons (الهلالين). شمال: SVM خطي على الـ features الأصلية (دقة 0.86). يمين: بعد إضافة polynomial features من الدرجة 3 الحد الفاصل بقى منحني (دقة 0.99).

انتبه / تصحيح

ملحوظة من التجربة الفعلية: `max_iter=10000` **مش ضروري** في الحالة دي: جربت الموديل وطلع إنه اتقارب في **10 تكرار بس** (ولما رفعت الـ degree لـ 10 احتاج 31 تكرار)، يعني الـ default (1000) كان كفاية ومطلعش أي ConvergenceWarning. الكتاب بيحطها كاحتياط. القاعدة: لو ظهرلك `ConvergenceWarning` ارفع `max_iter`، ولو مظهرش مفيش داعي.

وبالنسبة لعدد الـ features: على الداتا دي (features، degree 3 2) بقى عندنا **10** features (بما فيها الـ bias). لكن لو عندك 100 feature و degree 3 بتبقى **176,851**، ولو degree 10 تبقى **46,897,636,623,981** (47 تريليون!). ده بالظبط سبب إننا محتاجين الـ kernel trick.

7 Kernel Trick و Polynomial Kernel

من كشكولك

يعني إيه Kernel Trick؟ تعالى نبني الفكرة بالتدريج: إحنا عايزين نضيف Features جديدة تساعدنا نفصل البيانات غير الخطية، لكن أحيانًا التحويل الصريح للبيانات ينتج عددًا ضخمًا أو حتى **عددًا لا نهائيًا** من الأبعاد في بعض التحويلات.

→ الـ Kernel Trick يسمح للـ SVM بالحصول على **نتيجة مماثلة** لما يكون الوصول إليه باستخدام تحويل Features معيّن **من غير ما نحتاج نحسب التحويل نفسه صراحةً**.

→ الفكرة: تعتمد على استخدام **دالة تحسب علاقة رياضية بين عينتين**، بدل ما تنشئ كل الـ Features الجديدة.

إضافة

$$K(\mathbf{a}, \mathbf{b}) = (\gamma\, \mathbf{a}^T \mathbf{b} + r)^d \qquad (d = \text{degree},\ r = \text{coef0})$$

ده الـ polynomial kernel: بدل ما نحسب `PolynomialFeatures` ونضرب، بنحسب `(γ·aᵀb + r)^d` مباشرة بين العينتين (بس `a·b` على الـ features الأصلية). نتيجته نفس تأثير إضافة كل الـ features حتى الدرجة `d`، من غير ما نعمل الـ 47 تريليون عمود.

من كشكولك

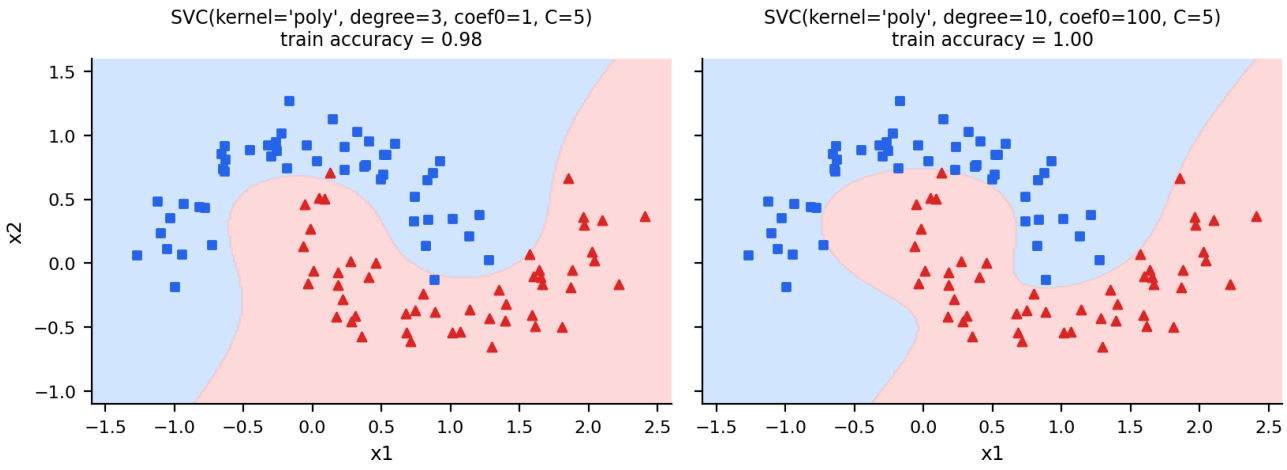
مثال Polynomial kernel:

```
from sklearn.svm import SVC
poly_kernel_svm_clf = make_pipeline(StandardScaler(),
                                    SVC(kernel="poly", degree=3, coef0=1, C=5))
poly_kernel_svm_clf.fit(X, y)
print(poly_kernel_svm_clf.score(X, y))
```

```
Output (real run)
0.98      # support vectors = 19 of 100
```

من كشكولك

- الـ Polynomial kernel بيستخدم `SVC(kernel="poly", degree=3, coef0=1, C=5)`.
- `coef0` → **بيتحكم في تأثير الحدود الأقل درجة بالمقارنة بالحدود الأعلى.**
- لو الموديل بيعمل overfitting قد يكون من المناسب **تقليل درجة** الـ Polynomial kernel، ولو بيعمل underfitting ممكن تجربة درجة أعلى، **مع التقييم** بدل افتراض إن الزيادة هتحسّن الأداء دائمًا.



تشغيل حقيقي: شمال: degree=3, coef0=1, C=5 (دقة 0.98، 19 support vectors). يمين: degree=10, coef0=100, C=5 (دقة 1.00، 8 support vectors): منحنى أعقد بكتير. دقة التدريب 1.00 هنا علامة تحذير: ممكن overfitting، والحكم بالـ validation مش بالـ training.

8 Gaussian RBF Similarity Features

من كشكولك

Similarity Features هي خاصية تعبّر عن مدى تشابه عينة معينة مع شيء آخر (زي نقطة مرجعية). تخيّل إنك بتبني model توقع أسعار البيوت وعايز تضيف Feature (مش موجودة في الداتا) تشوفلك: **الشقة دي قريبة قد إيه من وسط المدينة؟**

الفكرة هنا: بدل ما نكتفي بالـ Features الأصلية، نُنشئ Features تمثّل مدى تشابه كل عينة مع نقاط مرجعية اسمها landmarks.

→ الـ **Gaussian RBF Similarity Feature** تاخد الصورة الآتية:

$$\phi_\gamma(\mathbf{x}, \ell) = \exp\left(-\gamma \|\mathbf{x} - \ell\|^2\right)$$

(ℓ = الـ landmark، x = العينة، $\|x - \ell\|^2$ = مربع المسافة بينهما، γ تتحكم في سرعة تناقص التشابه مع المسافة.)

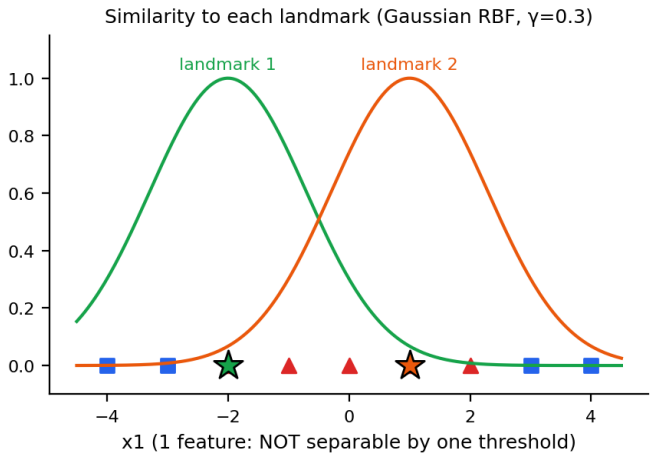
→ القيمة تكون **قريبة من 1** لما العينة تكون قريبة من الـ landmark وتقترب من **الصفر** كلما ابتعدت عنه، وبالتالي كل عينة ممكن تحصل على Similarity Feature لكل landmark.

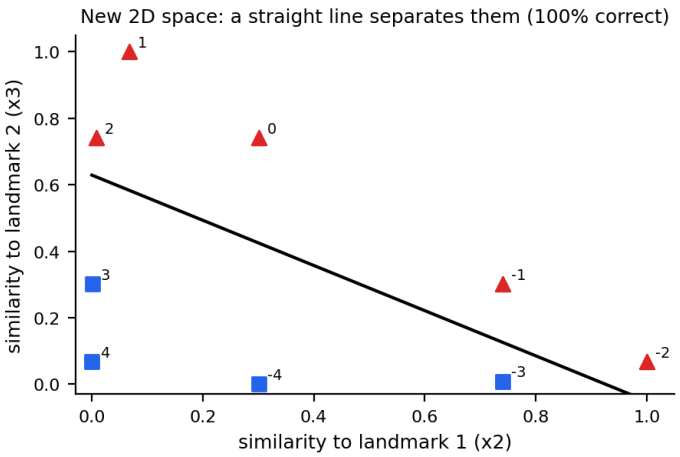
```
import numpy as np
def rbf(x, landmark, gamma): return np.exp(-gamma * (x - landmark) ** 2)

x1 = np.arange(-4, 5)                       # 9 points, 1 feature:  -4 ... 4
y  = [0, 0, 1, 1, 1, 1, 1, 0, 0]            # the middle points are class 1 -> NOT separable by one
threshold
F  = np.c_[rbf(x1, -2, 0.3), rbf(x1, 1, 0.3)]     # 2 similarity features (landmarks at -2 and 1)
print(F[2].round(3))                        # the point x1 = -2  (sits exactly on landmark 1)
print(F[3].round(3))                        # the point x1 = -1
```

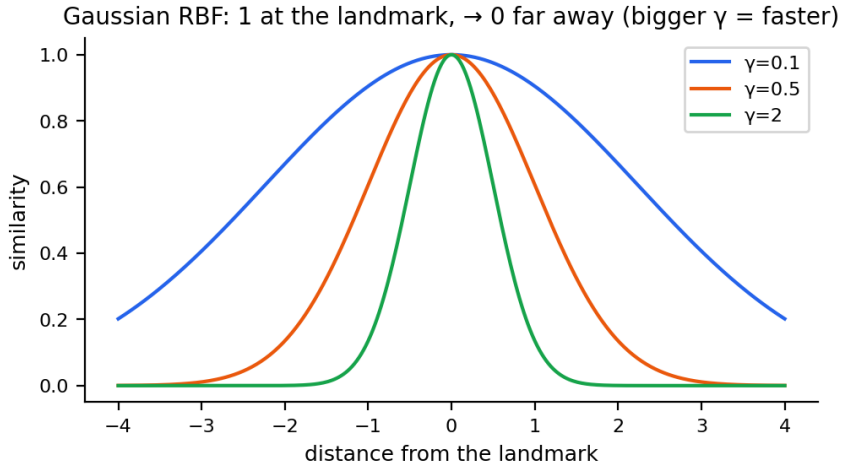
Output (real run)

```
[1.000 0.067]
[0.741 0.301]
```





نفس مثال الكتاب بتشغيل حقيقي: شمال: بُعد واحد، والنقط الحمرا (الوسط) محاطة بالزرقا من الناحيتين، فمفيش threshold واحد يفصلهم (دقة خط واحد 0.56). يمين: بعد تحويل كل نقطة لـ similarity مع landmarkاين، بقت 2D وخط مستقيم بيفصلهم 100% صح.



منحنى الـ RBF: 1 عند الـ landmark، وبيقل مع المسافة. `γ` أكبر = أضيق وأسرع هبوط.

من كشكولك

→ **المشكلة:** لو إنشاء Landmark لكل عينة ممكن يزوّد عدد Features بشكل كبير، فلو عندك `m` عينة قد تحتاج `m` Features جديدة. والـ Kernel Trick يحاول تحقيق **تأثير مشابه من غير إنشاء كل هذه الـ Features صراحةً**.

9 Gaussian RBF Kernel و Gamma

من كشكولك

نفس القانون بتاع Similarity Feature (Gaussian) ونفس كل حاجة تقريبًا، بس الفرق إن RBF kernel بيحسب العلاقة بين عينتين باستخدام **المسافة بينهما**، ومش لازم تحدّد يدويًا landmarks وبتنشئ الـ Similarity Features بنفسك.

$$K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma \|\mathbf{a} - \mathbf{b}\|^2\right)$$

```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(),
                                   SVC(kernel="rbf", gamma=5, C=0.001))
rbf_kernel_svm_clf.fit(X, y)
print(rbf_kernel_svm_clf.score(X, y))
```

```
Output (real run)
0.97
```

من كشكولك

→ لما استخدمنا `kernel="rbf"` يعني **Gaussian Radial Basis Function**.

إضافة

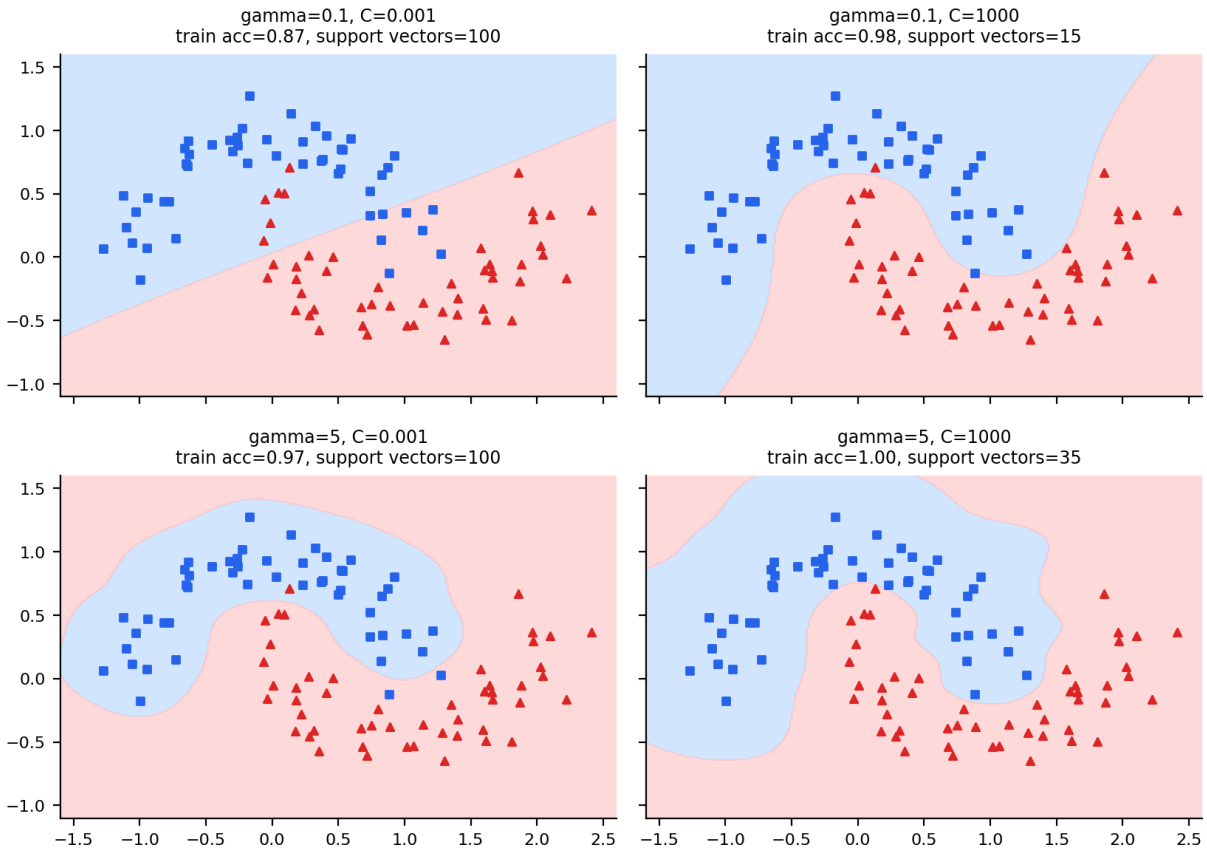
توضيح بسيط لجملتك "مش لازم تحدد landmarks": الـ RBF kernel عمليًا بيعامل **كل نقطة تدريب كـ landmark ضمني** (الكتاب بيقول كده)، بس من غير ما يبني الـ m عمود فعلًا. وده سر إن `SVC` بتبطأ لما m تكبر (القسم 5): الـ kernel بيتحسب بين كل زوج من النقط.

من كشكولك

⇒ **الـ Hyperparameter `gamma` :**

- **لو `gamma` زادت:** (1) تأثير كل عينة يصبح **أكتر محلية** (التشابه بيقل بسرعة) (2) الحد الفاصل يصبح **أكتر تعرجًا** (3) يزيد خطر overfitting.
- **لو `gamma` قلت:** (1) تأثير كل عينة يمتد لمسافة **أكبر** (2) الحد الفاصل يصبح **أنعم** عادةً (3) إذا قلت أكتر من اللازم ممكن يحصل underfitting.

⇒ إذن `gamma` **تتحكم في مدى تأثير العينة على شكل الحد الفاصل**، وتؤثر على تعقيد الموديل. ⇒ ولذلك الكتاب يعتبرها شبيهة بـ Regularization Hyperparameter.

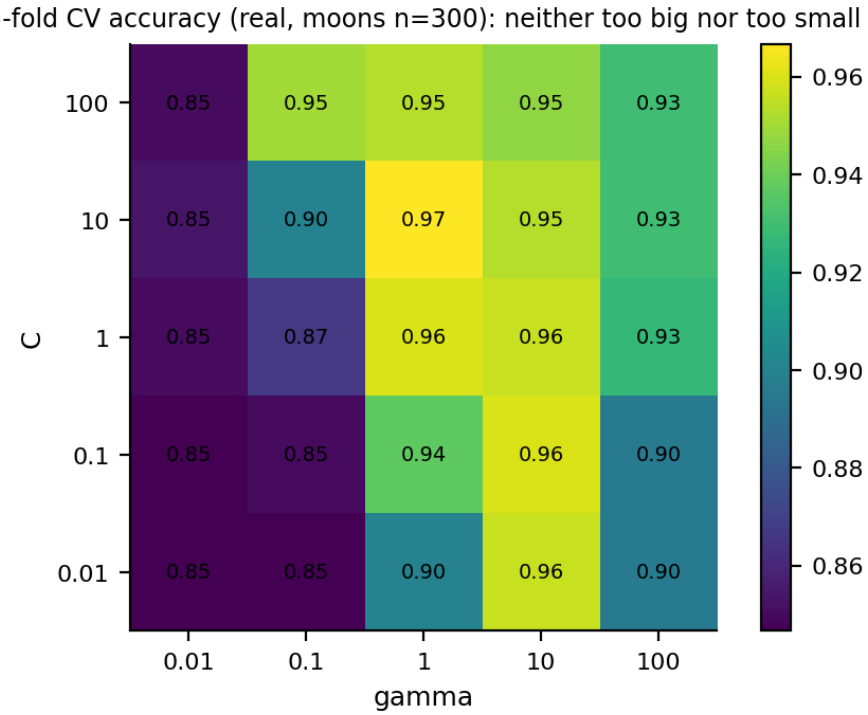


تشغيل حقيقي لتأثير `gamma` و`C` على الـ moons (نفس القيم اللي في الكتاب). فوق (gamma=0.1 صغيرة): الحد أنعم. تحت (gamma=5 كبيرة): الحد معقد. و`C=0.001` (يسار) بيدّي موديل مقيّد (100 support vector)، و`C=1000` (يمين) بيحاول يطابق كل نقطة.

إضافة

gamma	C	دقة التدريب	عدد الـ support vectors
0.1	0.001	0.87	100
0.1	1000	0.98	15
5	0.001	0.97	100
5	1000	1.00	35

لاحظ: `C=0.001` بيخلّي **كل** النقط (100 من 100) support vectors لأن الـ margin واسع جدًا ومعظم النقط جواه؛ و`C=1000` بيقلّلهم لـ 15 أو 35. وده منطقي مع كلامك: `C` صغيرة = مرونة أكتر = نقاط أكتر جوه الـ margin.



الاختيار الصح بالـ cross-validation (5 folds، داتا moons أكبر): أحسن توليفة طلعت C=10, gamma=1 بدقة 0.967. القيم الصغيرة جدًا (يسار/تحت) بتعمل underfitting، والكبيرة جدًا بتنزل الدقة شوية (overfitting).

```
from sklearn.model_selection import GridSearchCV
param_grid = {"svc__C": [0.01, 0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(make_pipeline(StandardScaler(), SVC(kernel="rbf")), param_grid, cv=5).fit(X, y)
print(grid.best_params_, round(grid.best_score_, 3))
```

```
Output (real run)
{'svc__C': 10, 'svc__gamma': 1} 0.967
```

10 الـ SVM Regression

من كشكولك

هل الـ SVM خاص بالـ Classification فقط؟ لا، يمكن استخدامه أيضًا مع Regression.

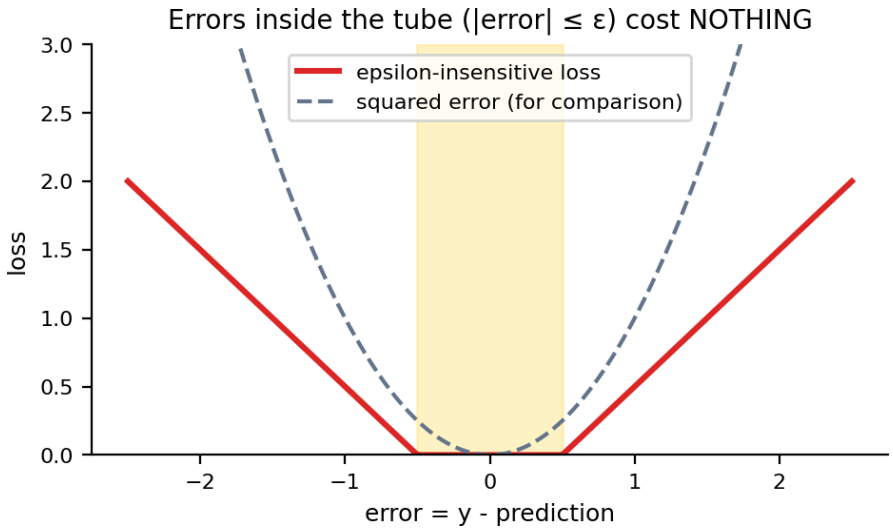
→ فاكر في الـ Classification كنا بنحاول نرسم **أكبر شارع** ممكن بين فئتين؟ في SVM Regression **الهدف مختلف:** بنحاول إيجاد دالة تمرّر **أكبر عدد ممكن من العينات داخل شارع** عرضه يحدده الـ Hyperparameter ε (epsilon) مع تقليل عقوبة العينات الخارجة عنه.

→ لو العينة تنبؤها **داخل نطاق** ε حول القيمة المستهدفة لا تتحمّل عقوبة من نوع Epsilon-Insensitive loss لمجرد وجود خطأ داخل هذا النطاق.

→ والعينات الواقعة **خارج النطاق** هي التي تساهم في الخطأ وفق مقدار ابتعادها، مع مراعاة إعدادات الموديل.

إضافة

$$\ell_\varepsilon(y, \hat{y}) = \max(0,\ |y - \hat{y}| - \varepsilon)$$



الـ loss بتاعة SVR: الأصفر هو الـ tube (الخطأ جواه بصفر). برا الـ tube الخطأ بيتحسب خطي (الأحمر)، عكس الـ squared error (المتقطع).

من كشكولك

- `ε` **كبيرة:** نطاق التسامح أوسع، وبالتالي يمكن أن تبقى عينات أكتر داخل النطاق دون عقوبة.
- `ε` **صغيرة:** نطاق التسامح أضيق، وبالتالي يصبح عدد أكبر من العينات مؤثرًا في تحديد الحل.

→ الكتاب يشير إلى: إن تقليل الـ `ε` يزيد عدد الـ Support Vectors، وإن العينات التي تظل **داخل الـ Margin** (الشارع) لا تؤثر في التنبؤات بالطريقة نفسها التي تؤثر بها العينات الواقعة خارجها.

من كشكولك

Linear SVM Regression

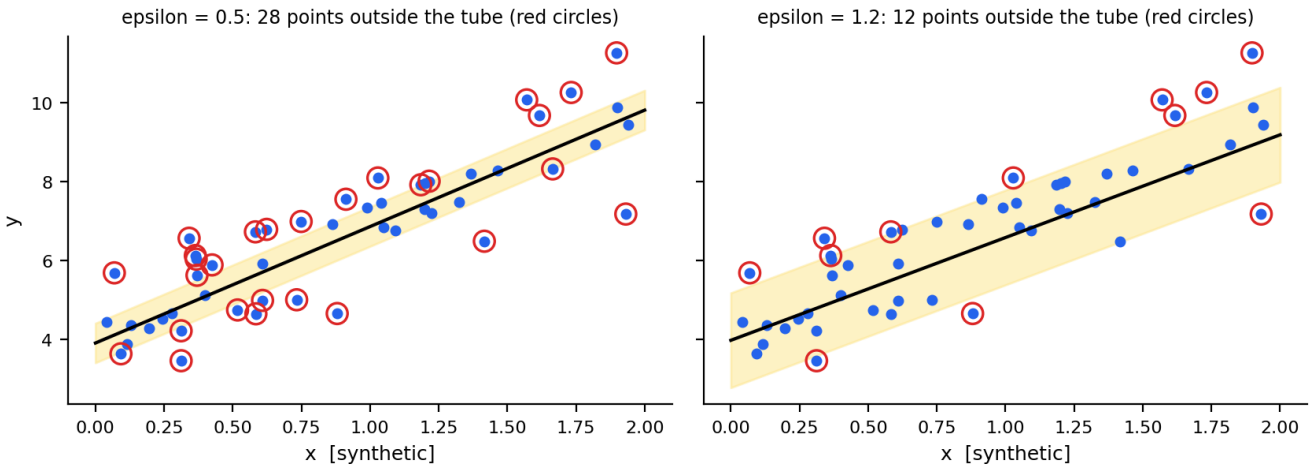
```
from sklearn.svm import LinearSVR
svm_reg = make_pipeline(StandardScaler(), LinearSVR(epsilon=0.5, random_state=42))
svm_reg.fit(X, y)
print(svm_reg.predict([[1.0]]))
```

Output (real run)

```
[6.8566]      # (synthetic data: y = 4 + 3x + noise  ->  about 7 at x = 1)
```

من كشكولك

الفكرة: `Features` → `StandardScaler` → `LinearSVR` → `Predicted Numeric Value` . ده يُستخدم Linear SVM Regression.



تجربة حقيقية: تقليل `ε` من 1.2 لـ 0.5 خلّى الـ tube أضيق، فالنقط اللي خارجه (الدوايز الحمرا) زادت من 12 لـ 28، وده بالظبط معنى "`ε` صغيرة = عدد أكبر من العينات مؤثر".

من كشكولك

Nonlinear SVM Regression

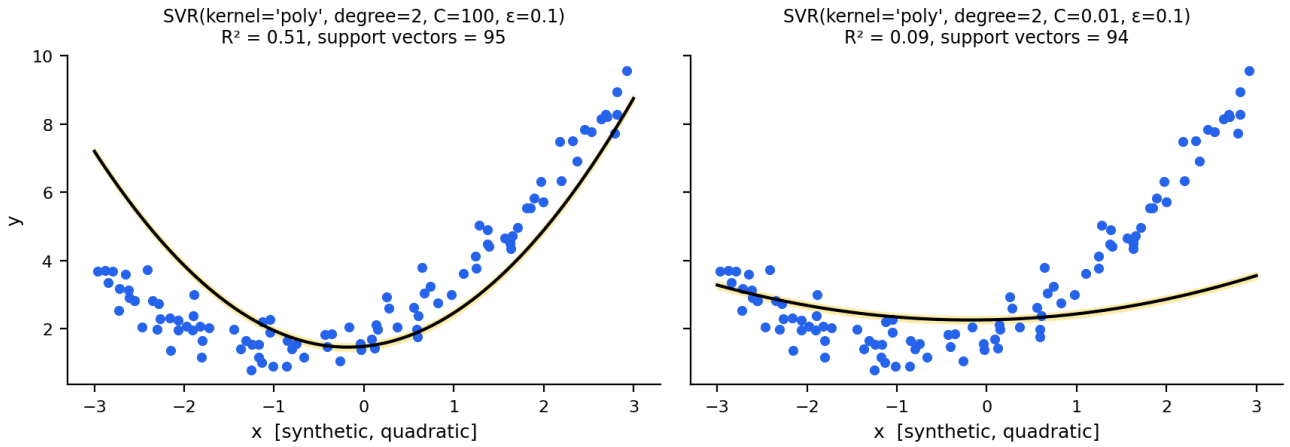
لو العلاقة غير خطية، يمكن استخدام `SVR` مع kernel:

```
from sklearn.svm import SVR
svm_poly_reg = make_pipeline(StandardScaler(),
                             SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1))
svm_poly_reg.fit(X, y)
print(svm_poly_reg.predict([[1.5]]))
```

```
Output (real run)
[2.6203]      # (synthetic quadratic data: y = 0.5x² + x + 2 + noise  ->  about 4.6 at x = 1.5)
```

من كشكولك

→ هنا `SVR` نسخة الـ SVM الخاصة بالـ Regression التي تدعم Kernel Trick. → الكتاب يوضّح إن `SVR` قد يصبح **بطيئًا** مع عدد عينات ضخم، بينما `LinearSVR` **يتوسع خطيًا تقريبًا** مع حجم البيانات.



الداتا (quadratic (synthetic. شمال: C=100 الـ SVR تتبّع المنحنى بشكل معقول. يمين: C=0.01 الموديل مقيّد جدًا (flat) وR² بس 0.09. نفس منطق الـ `C` في الـ classification: `C` صغيرة = regularization قوية.

إضافة

انتبه للقيمة اللي طلعت في التجربة: الـ prediction للمدخل 1.5 = **2.62** بس، مع إن الداتا الحقيقية عند 1.5 حوالي 4.6: السبب إن `C=0.01` بيخلّي الموديل مقيّد جدًا، و`coef0` الـ 0 = default فالـ kernel ملوش حدود من الدرجة الأولى. لو ضفنا `coef0=1` (زي ما اتعلمنا في القسم 7) مع `C=100` بيطلع R^2 = **0.96** بدل 0.51. يعني `coef0` و`C` مهمين للـ SVR زي ما همّوا للـ SVC.

وملحوظة أخيرة: الـ SVM للـ **Novelty detection** (اللي ذكرته في أول الشابتر) اسمه `OneClassSVM` وبيتشرح في شابتر 9.

11 أسئلة مراجعة سريعة

جاوب الأول في دماغك، وبعدين شوف الإجابات في الصفحة اللي بعدها.

1. إيه هو الـ SVM؟ وفيه استخدم إيه (3 استخدامات)؟
2. إمتى SVM بيكون مفيد؟ وإمتى بيبقى بطيء؟
3. اشرح فكرة Large Margin classification. وإيه الفرق بينها وبين أي خط بيفصل الفئتين؟
4. إيه الـ Decision Boundary والـ Margin والـ Support Vectors؟
5. إيه اللي بيحصل لو أضفنا نقط بعيدة عن الـ margin؟ وإيه لو أضفنا نقطة على الـ margin؟
6. ليه الـ SVM حساس لمقياس الـ features؟ وإيه الفرق في التأثير بين linear و rbf؟
7. إيه هو الـ Hard Margin؟ وإيه مشكلتيه؟
8. إيه هو الـ Soft Margin؟ وإيه هو ج؟
9. إيه دور الـ C؟ وإيه اللي بيحصل لو صغّرناه أو كبّرناه؟
10. هل C أكبر معناها موديل أفضل؟ وإزاي نختاره؟
11. ليه بنستخدم random_state؟
12. إيه معنى decision_function في SVM؟ وهل هي probability؟
13. ليه LinearSVC مفيهاش predict_proba؟ وإيه الحل؟
14. إيه خطوات الـ pipeline: PolynomialFeatures + StandardScaler + LinearSVC؟ وإيه مشكلتها لما الـ degree يكبر؟
15. إيه هو الـ Kernel Trick؟
16. إيه هو الـ coef0 في الـ polynomial kernel؟
17. إيه هي الـ Similarity Features والـ landmarks؟ واكتب معادلة Gaussian RBF.
18. إيه الفرق بين RBF similarity features و RBF kernel؟
19. إيه تأثير gamma لما تكبر أو تصغر؟
20. ليه الكتاب بيشبّه gamma بـ regularization hyperparameter؟
21. إزاي الـ SVM Regression بيشتغل؟ وإيه دور ε؟
22. إيه اللي بيحصل لعدد الـ support vectors لما نقلل ε؟
23. إيه الفرق بين LinearSVR و SVR؟
24. إيه الفرق في التعقيد بين LinearSVC و SVC و SGDClassifier؟

12 إجابات أسئلة المراجعة

1. Support Vector Machine: موديل قوي بيُستخدم في Classification و Regression و Novelty Detection.
2. مفيد مع الـ datasets الصغيرة والمتوسطة وخاصة الـ non-linear classification، وبطيء (لا يتوسع) مع عدد عينات ضخم (SVC بالذات).
3. بدل أي خط بيفصل الفئتين، بنختار الخط اللي بيعمل أكبر مسافة (margin) بينه وبين أقرب نقاط من الفئتين، فيبقى أقل حساسية لتغيّر بسيط في الداتا.
4. Decision boundary: الحد الفاصل. Margin: الشارع حواليه. Support vectors: النقاط اللي على حافة الـ margin أو جواه وهي اللي بتحدد الحل.
5. النقط البعيدة عن الـ margin مبتغيّرش الحل (جربناها: نفس w و b). نقطة على/جوه الـ margin بتبقى support vector جديدة وبتغيّر الحل.
6. لأن الـ margin والـ kernel مبنيين على مسافات، فالـ feature الكبيرة بتسيطر. الأثر كان ضخم على rbf (0.49 → 0.90) وصغير على linear (0.90 → 0.91)، بس الأفضل scaling دايمًا.
7. كل النقط لازم تبقى برا الـ margin وفي الجانب الصح. مشكلتيه: لا حل لو الداتا مش قابلة للفصل الخطي، وحساس جدًا للـ outliers (margin بينهار).
8. نسمح ببعض الانتهاكات (نقط جوه الـ margin أو في الجانب الغلط). ζ = مقدار انتهاك كل عينة، والهدف توازن بين margin كبير وانتهاكات قليلة.
9. C صغيرة: انتهاكات أكتر + margin أعرض + regularization أقوى (خطر underfitting). C كبيرة: عقاب أقوى للانتهاكات + margin أضيق (خطر overfitting).
10. لأ. الاختيار الأنسب بيعتمد على الداتا ونتايج التقييم (cross-validation / GridSearchCV).
11. عشان العشوائية (خلط/تهيئة) تبقى ثابتة، فأي تغيّر في النتيجة يبقى سببه تغيير الـ hyperparameter مش العشوائية.
12. score = مقياس لموقع العينة من الحد الفاصل (موجب = Positive، سالب = Negative، قريب من 0 = قريبة من الحد). مش probability.
13. لأنه مش بيقدّر احتمالات بشكل مباشر. الحل: SVC(probability=True) اللي بيضيف نموذج إضافي لتقدير الاحتمالات (وبيزود وقت التدريب).
14. تضيف features بدرجة 3 ثم scaling ثم linear SVM. مشكلتها: عدد الـ features ينفجر مع الـ degree وعدد الـ features الأصلية (بطء وذاكرة).
15. طريقة نحصل بيها على نتيجة مماثلة لإضافة features كتيرة (حتى لا نهائية) من غير ما نحسبها صراحة، بدالة kernel بتحسب العلاقة بين عينتين.
16. بيتحكم في تأثير الحدود الأقل درجة بالنسبة للحدود الأعلى. في الـ SVR التجربة بيّنت إن coef0=0 بيحرم الموديل من الحدود الخطية.
17. similarity feature = مدى تشابه العينة مع نقطة مرجعية (landmark): 1) $\exp(-\gamma\|x-\ell\|^2)$ عند الـ landmark ويقل بالمسافة).
18. similarity features: بنبني m عمود صراحةً (landmarks). RBF kernel: بيحسب التشابه بين عينتين مباشرة من غير ما يبني الأعمدة.

19. gamma كبيرة: تأثير كل عينة محلي، الحد متعرج، خطر overfitting. gamma صغيرة: تأثير ممتد، الحد أنعم، خطر underfitting.
20. لأن gamma بتتحكم في تعقيد الموديل (زي regularization): gamma أقل = موديل أبسط وأنعم.
21. بيحاول يحط أكبر عدد من العينات جوه شارع عرضه ε؛ الخطأ جواه بدون عقوبة، والعقوبة بس للعينات اللي برا بحسب بُعدها.
22. عدد الـ support vectors بيزيد (نقط أكتر بتبقى برا الـ tube).
23. LinearSVR خطي وبيتوسع تقريبًا خطيًا مع m. SVR بيدعم kernels لكن بيبطأ مع m الكبير.
24. LinearSVC: O(m×n) بدون kernel trick. SGDClassifier: O(m×n) مع وبيدعم out-of-core. SVC: O(m²×n) لـ O(m³×n).